DB connection pool

A database connection pool is a managed collection of open, reusable database connections. Instead of creating a brand-new connection every time a Python application needs to run SQL, the app borrows an available connection from the pool, uses it, then returns it for the next request.

Connection pooling improves performance because opening a database connection can involve network communication, authentication, session setup, and server resource allocation. Reusing existing connections reduces that repeated overhead and limits the total number of concurrent connections the database must manage

Without a pool

Request → Open DB connection → Run SQL → Close connection

Request → Open DB connection → Run SQL → Close connection

With a pool

Request → Borrow connection → Run SQL → Return connection

Request → Borrow connection → Run SQL → Return connection

A database server has a finite connection limit and must allocate memory, processes/threads, and scheduling work to active connections. If every web request creates its own connection, an application can become slow or fail during a traffic spike. A pool acts as a controlled waiting area: it reuses a limited number of connections and queues requests when all available connections are busy.

For example, a Python web application might receive 100 simultaneous requests but use a pool that allows only 10 database connections. At most 10 requests perform database work at the same time; the other requests wait briefly for a connection to be returned.

Setting	Meaning	Performance consideration
Pool size	Number of persistent reusable connections kept by the application	Increase only when the database has capacity and connections are the actual bottleneck
Max overflow	Temporary extra connections allowed when the main pool is fully used	Keep limited; high overflow can shift overload to the database
Pool timeout	How long a request waits for a connection before failing	Use a finite, user-appropriate value; prevents requests from waiting forever
Recycle / max lifetime	How long a connection can remain open before replacement	Set below infrastructure/database idle timeouts when connections may be dropped
Pre-ping / health check	Verifies that a borrowed connection is still usable	Adds a small check but helps prevent failures from stale connections
Minimum idle connections	Connections maintained ready for new work, if supported	Can reduce cold-start latency but consumes database capacity while idle

Size the pool across the whole application

Do not choose a pool size by looking at one Python process alone. Calculate the maximum possible database connections across all running application instances:

Optimize query time before increasing pool size

Pool exhaustion often means connections are being held too long, commonly because queries are slow, blocked, or run inside long transactions. Adding more connections may worsen contention and database load instead of solving the root cause.

Return connections promptly

Use a context manager or try/finally pattern so that a connection is returned even when an error occurs.

Use timeouts and resilient connection handling

Set a reasonable acquisition timeout. A request that waits indefinitely for a database connection consumes application resources and provides a poor user experience.

Measure the pool itself

A well-instrumented application should collect:

- Connections checked out/in use.
- Idle connections.
- Pool wait time: time spent acquiring a connection.
- Pool timeout/failure count.
- Query duration and database error rate.
- Active database connections and database connection limit.
-

High API latency
+ low SQL execution time
+ high pool acquisition wait time
= likely pool contention, leaked connections, or undersized pool

High API latency
+ high SQL execution time
+ high pool acquisition wait time
= queries are holding connections too long;
optimize SQL before enlarging the pool

What too many connections cost

Resource	What happens as connection count rises	Result
Memory	Each session needs memory and connection-management resources	Less RAM remains for database cache and query work
CPU	More active sessions compete for CPU time	More scheduling/context-switching overhead
Disk I/O	Many queries may read different tables/indexes at once	More queueing and random I/O; slower queries
Locks	More concurrent transactions touch the same rows/tables	Lock waits, deadlocks, and retries can increase
Connection setup	A large pool may keep many idle sessions open	Wasted capacity and more operational overhead
Failure behavior	A traffic spike can push all connections into the database	The database becomes overloaded instead of applying backpressure

What is observability?

Observability is the ability to understand what is happening inside a system by examining the data it produces. It lets developers answer not only “Is the system working?” but also “Why is it slow, failing, or behaving unexpectedly?”—including for problems they did not anticipate

Observability means being able to investigate questions such as:

- Why is the /orders or /students/{id}/schedule endpoint slow?
- Did the delay occur in Python code, while waiting for a database connection, or during a SQL query?
- Which query failed, timed out, or returned unexpected results?
- Did performance worsen after data volume increased or an index changed?

Database Observability

- Distinguish monitoring, telemetry, and observability.
- Explain logs, metrics, and traces as complementary telemetry signals.
- Identify database and application measurements that reveal performance, errors, and data-access behavior.
- Instrument a Python application that uses a relational database.
- Define useful metrics without exposing sensitive data or creating unmanageable metric cardinality.
- Interpret a trace to locate an application or database bottleneck.

Why observability belongs in database implementation

A database can be logically correct yet operationally unhealthy.

- A query may return the correct rows but take 8 seconds.
- A connection pool may be exhausted during peak traffic.
- A missing index may cause table scans as data grows.
- A transaction may deadlock or wait on a lock.
- An application may retry failed writes, creating duplicate requests or excess load.

When a user reports “the system is slow,” can we explain what happened, where it happened, and why?

Term	Plain-language meaning	Example
Monitoring	Watching known signals and alerting when a threshold is crossed	Alert when DB connection usage exceeds 80%
Telemetry	Operational data emitted by systems	A query-duration measurement or an error log
Observability	Ability to infer internal system state from emitted signals	Determine that checkout latency comes from one slow SQL query
Instrumentation	Code/configuration that produces telemetry	A Python timer around `cursor.execute()`

Observability is not a dashboard. Dashboards are one way to view the data; observability is the ability to investigate the system's behavior using its telemetry

Three signals

Signal	Best for answering	Typical database/app content
Logs	“What happened?”	Error message, request ID, SQL state, exception stack trace
Metrics	“How much/how often/how long?”	Request rate, p95 query latency, errors, active connections
Traces	“Where did time go across this request?”	HTTP request → Python function → SQL query → response

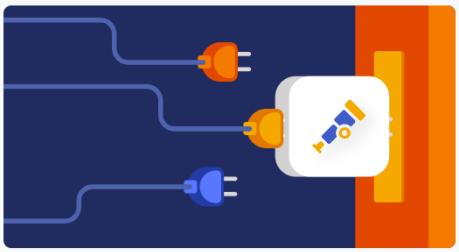
A fourth useful category is **profiles**: periodic samples of CPU, memory, or code-level execution.

KubeCon + CloudNativeCon North America 2026 • Nov 9–12 • Salt Lake City • Details

OpenTelemetry is an open source observability framework for cloud native software. It provides a single set of APIs, libraries, agents, and collector services to capture distributed traces and metrics from your application.

OpenTelemetry builds upon years of experience from the OpenTracing and OpenCensus projects, combined with best-of-breed ideas and practices from the community.





Vendor-neutral instrumentation

Instrument your code once using OpenTelemetry APIs and SDKs. Export telemetry data to any observability backend—Jaeger, Prometheus, commercial vendors, or your own solution. Switch backends without touching your application code.

Example: course registration

Assume a Flask-style Python app exposes this endpoint:

```
GET /students/1042/schedule
```

The endpoint:

1. Receives the HTTP request.
2. Validates `student_id`.
3. Queries `enrollment`, `section`, and `course` tables.
4. Formats JSON.
5. Returns the schedule.

If the endpoint is slow, telemetry should help answer:

- Is the slowness in Python or MySQL/PostgreSQL?
- Which SQL operation took time?
- Did the query error, time out, or wait for a connection?
- Does it happen for every request or only some requests?
- Did the issue begin after a release or schema/index change?

A student uses a Python course-registration app and reports that the schedule page takes 4 seconds to load.

Observability could show:

```text
GET /students/1042/schedule     4.0 seconds
├── Python validation           0.02 seconds
├── Wait for DB connection      2.5 seconds
├── SQL SELECT query            1.4 seconds
└── Format JSON response        0.08 seconds
```

This evidence indicates that the main issue is not JSON formatting or application logic. The app is waiting for a database connection, and the SQL query also needs investigation. The developer can then examine connection-pool settings, concurrent requests, query plans, indexes, and database-server load.

Observability vs. monitoring

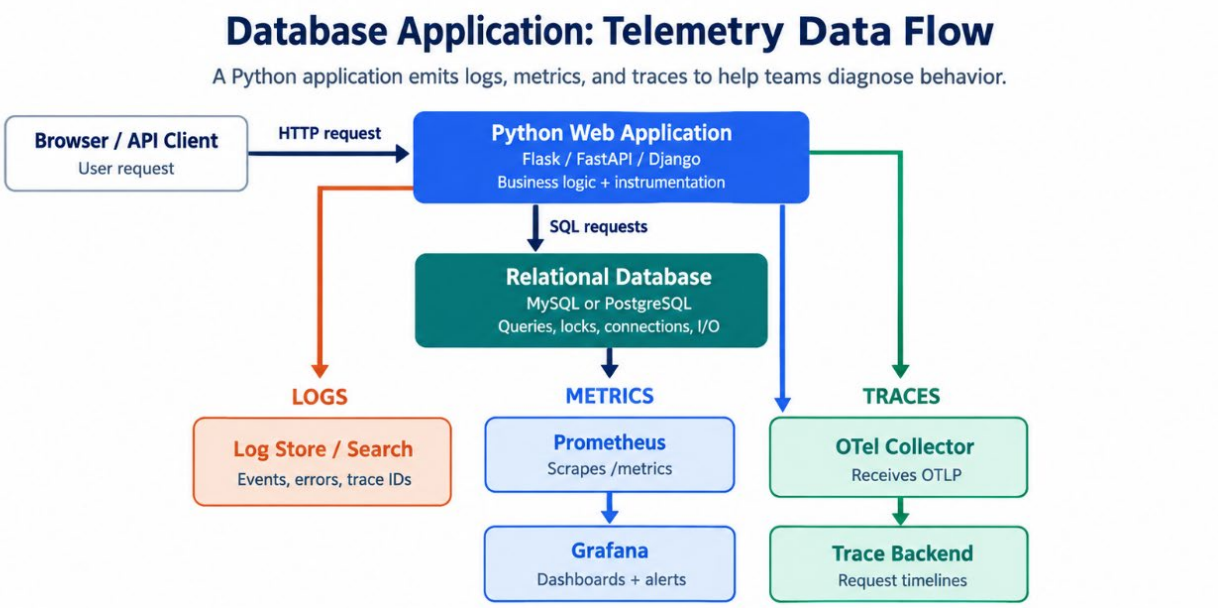
Monitoring watches predefined conditions, such as “alert when database CPU exceeds 85%” or “alert when API errors exceed 2%.”

Observability supports investigation: it combines telemetry so you can ask new questions, determine root causes, and explain why a problem occurred—even when no specific alert was designed for it.

Key takeaway

Observability turns system behavior into evidence that developers can use to diagnose, explain, and improve an application.

For the database implementation project, observability is part of a complete solution: the application should not only store and retrieve data correctly; it should also expose enough telemetry to show whether it is fast, reliable, secure, and scalable.



Trace ID: A unique value shared by all related work performed for one request. It connects logs, metrics, and trace spans so developers can follow a request from the browser through the application and database

```
GET /students/1042/schedule
```

```
Trace ID: 4bf92f3577b34da6a3ce929d0e0e4736
```

```
Trace ID: 4bf92f3577b34da6a3ce929d0e0e4736
│
├── GET /students/1042/schedule                 2,430 ms
│   ├── validate_student_id                         5 ms
│   ├── acquire_database_connection                45 ms
│   ├── SELECT student schedule                 2,310 ms
│   │   └── MySQL: enrollment / section / course
│   └── build JSON response                        20 ms
│
└── Result: HTTP 200 OK
```

The trace shows that the overall request took 2.43 seconds and that the database query consumed 2.31 seconds. The investigation should therefore focus on the SQL statement, indexes, number of rows scanned, joins, and the query plan—not on JSON formatting.

Log entry


```
{
  "timestamp": "2026-09-28T15:50:12.411Z",
  "level": "WARNING",
  "event": "slow_database_query",
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "span_id": "00f067aa0ba902b7",
  "route": "/students/{student_id}/schedule",
  "operation": "SELECT",
  "database": "registration",
  "duration_ms": 2310,
  "rows_returned": 5
}
```


A developer can copy the trace_id from the log, search for it in a trace tool, and immediately view the entire request timeline. OpenTelemetry recommends recording trace_id and span_id as top-level fields in structured log formats such as JSON

Trace ID vs. span ID

Identifier	Purpose	Example
Trace ID	Identifies the entire request/workflow	`4bf92f3577b34da6a3ce929d0e0e4736`
Span ID	Identifies one operation within the workflow	`00f067aa0ba902b7`
Parent span ID	Connects a child operation to the operation that started it	The database-query span points back to the route-handler span

```
import logging
from opentelemetry import trace
logger = logging.getLogger(__name__)
tracer = trace.get_tracer(__name__)
def get_student_schedule(cursor, student_id):
    sql = """
        SELECT c.course_code, c.title, s.term
        FROM enrollment e
        JOIN section s ON e.section_id = s.section_id
        JOIN course c ON s.course_id = c.course_id
        WHERE e.student_id = %s
    """
    with tracer.start_as_current_span("get_student_schedule") as span:
        span.set_attribute("db.operation.name", "SELECT")
        span.set_attribute("db.namespace", "registration")
        trace_id = format(
            span.get_span_context().trace_id,
            "032x"
        )
        cursor.execute(sql, (student_id,))
        rows = cursor.fetchall()
        logger.info(
            "Schedule query completed",
            extra={
                "trace_id": trace_id,
                "event": "schedule_query_completed",
                "operation": "SELECT",
                "rows_returned": len(rows)
            }
        )
        return rows
```

What to measure

Application metrics

- HTTP request count, rate, error count, and latency.
- In-progress requests.
- Exceptions by broad type or route.
- Background-job successes, failures, and duration.

Database interaction metrics

- Query count by operation (`SELECT`, `INSERT`, `UPDATE`, `DELETE`).
- Query duration histogram.
- Query failures and timeouts.
- Connection acquisition time.
- Pool capacity, checked-out connections, and connection errors.
- Transaction rollback count and deadlock/lock-wait events.

Database-server metrics

- Active connections and maximum connections.
- Buffer/cache efficiency.
- Disk I/O, CPU, memory, and storage space.
- Rows read versus rows returned.
- Slow-query count and replication lag where applicable.

Metric types and design

Type	Use it for	Example
Counter	Events that only increase	`db_query_errors_total`
Gauge	A value that rises and falls	`db_pool_connections_in_use`
Histogram	Distribution of durations or sizes	`db_query_duration_seconds`
Summary	Client-side observations; less flexible aggregation	Request duration summary

For latency, prefer a **histogram** when you want percentile-style questions over time, such as “What is p95 query duration in the last 15 minutes?”

Logs: structured, correlated, safe

Unstructured log:

```
Error doing query for 1042
```

Structured JSON log:

```
{
  "level": "ERROR",
  "event": "database_query_failed",
  "operation": "SELECT",
  "table": "enrollment",
  "duration_ms": 1204,
  "error_type": "OperationalError",
  "trace_id": "9f31..."
}
```

Logging guidelines:

- Use consistent field names.
- Include timestamp, severity, service name, operation, and correlation/trace ID.
- Log error context, not secrets.
- Do not log passwords, access tokens, payment data, or sensitive personal data.
- Be cautious with raw SQL and bind parameters; values may contain sensitive data.

Traces: seeing one request end to end

A **trace** represents a single request or transaction across components. A **span** is one timed operation within that trace.

Example trace:

```
GET /students/1042/schedule            1.84 s
├── validate_student_id                0.01 s
├── acquire_db_connection              0.71 s
├── SELECT schedule                    1.03 s
│   └── database: PostgreSQL
└── serialize_json                     0.04 s
```

This trace suggests two questions:

1. Why did connection acquisition take 710 ms?
2. Why did the SQL query take 1.03 seconds?

The trace makes the investigation targeted

OpenTelemetry (OTel)

OpenTelemetry is a vendor-neutral framework for generating, collecting, and exporting telemetry, including traces, metrics, and logs. The Python implementation provides APIs, SDKs, exporters, and instrumentation libraries. Traces and metrics are listed as stable in the current Python documentation; logs are still marked development.

- A common model across Python, Java, JavaScript, .NET, Go, and more.
- Supports automatic instrumentation plus deliberate manual spans/metrics.
- Avoids tightly coupling application code to one dashboard vendor.
- Carries context from an HTTP request to downstream database calls.

Implementation stack

Component	Classroom option	Purpose
Application	Flask or FastAPI + Python	API/service students implement
Database	MySQL or PostgreSQL	Relational data and query workload
Tracing	OpenTelemetry SDK + console exporter	See spans locally first
Metrics	Prometheus Python client	Expose `/metrics` endpoint
Visualization	Prometheus + Grafana (optional Docker Compose)	Query and chart metrics
Production-style trace pipeline	OTel Collector + Jaeger/Tempo	Central trace storage and viewing

Start with the console exporter

Structure

```
observability-lab/
├── app.py
├── requirements.txt
├── schema.sql
├── load_test.py
└── docker-compose.yml        # optional extension
```

Example dependencies:

```
Flask
mysql-connector-python
prometheus-client
opentelemetry-api
opentelemetry-sdk
opentelemetry-exporter-otlp
opentelemetry-instrumentation-flask
opentelemetry-instrumentation-dbapi
```

The OpenTelemetry Python API and SDK are installed separately; exporter and instrumentation packages use corresponding package names.

Example schema

```
CREATE TABLE student (
  student_id INT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL
);

CREATE TABLE course (
  course_id INT PRIMARY KEY,
  course_code VARCHAR(12) NOT NULL,
  title VARCHAR(100) NOT NULL
);

CREATE TABLE section (
  section_id INT PRIMARY KEY,
  course_id INT NOT NULL,
  term VARCHAR(20) NOT NULL,
  FOREIGN KEY (course_id) REFERENCES course(course_id)
);

CREATE TABLE enrollment (
  student_id INT NOT NULL,
  section_id INT NOT NULL,
  enrolled_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (student_id, section_id),
  FOREIGN KEY (student_id) REFERENCES student(student_id),
  FOREIGN KEY (section_id) REFERENCES section(section_id)
);
```

Compare the schedule query before and after adding an index that supports the join/filter pattern.

Manual metrics in Python

```
from time import perf_counter
from prometheus_client import Counter, Histogram

DB_QUERIES = Counter(
   "db_queries_total",
   "Database queries attempted",
   ["operation", "status"]
```

```
)

DB_QUERY_SECONDS = Histogram(
  "db_query_duration_seconds",
  "Time spent executing database queries",
  ["operation"]
)

def execute_query(cursor, sql, params=(), operation="SELECT"):
  started = perf_counter()
  try:
    cursor.execute(sql, params)
    DB_QUERIES.labels(operation=operation, status="success").inc()
    return cursor.fetchall()
  except Exception:
    DB_QUERIES.labels(operation=operation, status="error").inc()
    raise
  finally:
    DB_QUERY_SECONDS.labels(operation=operation).observe(
      perf_counter() - started
    )
```

This records the count and distribution of query execution time while avoiding student IDs and raw SQL as metric labels.

Expose Prometheus metrics

```
from flask import Flask
from prometheus_client import make_wsgi_app
from werkzeug.middleware.dispatcher import DispatcherMiddleware

app = Flask(__name__)
app.wsgi_app = DispatcherMiddleware(app.wsgi_app, {
  "/metrics": make_wsgi_app()
})
```

Run the application, then visit:

```
http://localhost:5000/metrics
```

Prometheus client libraries expose internal application measurements through an HTTP endpoint that Prometheus can scrape. The Python client's quick-start example uses `pip install prometheus-client`, starts an HTTP server, and exposes timing/count data.

Manual traces in Python

```
from opentelemetry import trace

tracer = trace.get_tracer(__name__)

def get_student_schedule(cursor, student_id):
  sql = """
    SELECT c.course_code, c.title, s.term
    FROM enrollment e
    JOIN section s ON e.section_id = s.section_id
    JOIN course c ON s.course_id = c.course_id
    WHERE e.student_id = %s
  """

  with tracer.start_as_current_span("get_student_schedule") as span:
    span.set_attribute("app.student_lookup", True)
    span.set_attribute("db.operation.name", "SELECT")
    span.set_attribute("db.namespace", "registration")
    cursor.execute(sql, (student_id,))
    rows = cursor.fetchall()
    span.set_attribute("app.result_count", len(rows))
    return rows
```

Notice what is intentionally omitted:

- The student's name, email, or other sensitive data.
- The literal student ID as a metric label.
- A blanket capture of bind parameters.

DB-API automatic instrumentation

OpenTelemetry's DB-API instrumentation supports libraries that follow Python DB-API 2.0. When a framework or ORM-specific integration exists, use that first; otherwise, the DB-API integration can wrap a driver's `connect` function.

```
import mysql.connector
from opentelemetry.instrumentation.dbapi import trace_integration

trace_integration(mysql.connector, "connect", "mysql")

connection = mysql.connector.connect(
  host="localhost",
  user="app_user",
  password="change_me",
  database="registration"
)
```

Students should explain whether they used:

- Manual instrumentation.
- Automatic driver/framework instrumentation.
- Both, with manual spans for business operations and automatic spans for database calls.

SQL comments and correlation

Some database instrumentation can attach context to SQL using SQLCommenter. For example, it may append a `traceparent` comment to a query, helping link database log records back to a trace when database logging is enabled.

```
SELECT * FROM enrollment
WHERE student_id = ?
/* traceparent='00-01234567-abcd-01' */
```

Use this carefully:

- Verify support and operational behavior in your driver/database.
- Do not expose confidential context in comments.
- Do not assume SQL comments replace secure audit logging.

Trace setup: local first

```
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import (
    BatchSpanProcessor,
    ConsoleSpanExporter,
)

provider = TracerProvider()
provider.add_span_processor(
    BatchSpanProcessor(ConsoleSpanExporter())
)
trace.set_tracer_provider(provider)
```

Classroom workflow:

1. Start the app with the console exporter.
2. Make requests that run successful and failing queries.
3. Inspect emitted spans and their duration/status.
4. Add metrics.
5. Only then send telemetry to a collector/backend.

Reading telemetry: investigation example

Symptom: p95 API latency rises from 180 ms to 2.4 s after test data grows.

Signal	What students see	Likely next question
Metric	`db_query_duration_seconds` p95 increased for `SELECT`	Which operation or route is affected?
Trace	`GET /schedule` has a 2.1 s database child span	What SQL statement/query plan is responsible?
Log	Slow-query event shows a query fingerprint and trace ID	Does the plan use an index or scan many rows?

Database evidence	EXPLAIN shows a full scan	Which index supports the join/filter?

Possible fix: Add a carefully chosen index, rerun the same workload, and compare before/after p50 and p95 latency—not just one query's best-case result.

Alerts based on user impact

Avoid alerts that only say "CPU is high." Prefer alerts that identify a user-impacting condition.

Examples:

- Error rate: database query failures exceed 2% for 5 minutes.
- Latency: p95 schedule-endpoint latency exceeds 1 second for 10 minutes.
- Capacity: available pool connections remain near zero for 5 minutes.
- Reliability: transaction rollbacks or deadlocks increase above the normal baseline.

An alert should contain a clear condition, duration, service/route, severity, and link to the relevant dashboard or trace search.

Privacy, security, and cost

Telemetry is data. Treat it as part of the system's data governance.

- **Never** emit passwords, tokens, session IDs, or complete payment data.
- Avoid raw SQL parameter capture in classroom or production telemetry unless there is an approved, redacted need.
- Protect logs and dashboards with appropriate access control.
- Set retention policies: detailed traces are expensive at high traffic.
- Sample traces in production; preserve errors and slow traces at higher rates.
- Use query fingerprints/templates rather than one metric series per literal SQL statement.

Common mistakes

Mistake	Why it fails	Better approach
Logging every successful SQL statement with parameters	High volume; privacy exposure	Log slow/error events, redact values, use query fingerprints
Metric label includes student_id	Creates unbounded time series	Use route/operation/table labels; put IDs only in protected trace/log context if justified
Timing only HTTP requests	Cannot locate the slow layer	Trace or time major internal operations, including DB access
Alerting on every single error	Alert fatigue	Use rates, duration windows, and severity thresholds
Adding an index without measurement	May not improve the actual bottleneck	Compare query plans and latency distributions before/after
Treating telemetry as only a production concern	Bugs are harder to diagnose later	Include observability in the project's definition of done

Student project

Required deliverables

1. ERD and normalized relational schema.
2. Schema DDL and representative seed data.
3. Python application with at least four database-backed endpoints or operations.
4. Structured logs for requests and database errors.
5. At least three custom metrics, including:
    - One counter.
    - One histogram for a duration.
    - One gauge or application/database capacity indicator.
6. At least one trace containing an application span and a database operation span.
7. One intentionally introduced performance or reliability issue, such as a missing index, slow query, invalid foreign key write, or connection-pool saturation simulation.
8. Evidence-based diagnosis and a remediation, with before/after measurements.

9. A short README explaining how to run the app, generate traffic, and view telemetry.

Discussion questions

- What information belongs in logs versus metrics versus traces?
- Which latency statistic would you show a user-facing service owner?
- What telemetry would demonstrate that a new index actually improved the application?
- How could monitoring detect a connection-pool problem even if individual SQL queries are fast?
- Which telemetry fields would be unsafe to collect in a real student-information system?